

Caixeiro Viajante com Algoritmo Genético

Desenvolvido por: Kauan Adami, Nilson Neto
Professor: Rudimar Luis Scaranto



Estrutura Geral do Projeto

O projeto foi desenvolvido para resolver o problema do caixeiro viajante de forma interativa e visual, utilizando o algoritmo genético como método de busca de soluções otimizadas.

A aplicação foi construída em Python, integrando Tkinter para a interface gráfica e NumPy para o tratamento da matriz de distâncias.

O sistema permite ao usuário definir o número de cidades (até 20), preencher manualmente ou gerar automaticamente as distâncias, e acompanhar em tempo real a evolução do algoritmo



Modelagem do Problema

O problema foi tratado de forma a representar cada rota possível como um indivíduo dentro de uma população.

Cada indivíduo é composto por uma sequência de cidades que o caixeiro deve percorrer, sempre retornando à cidade inicial.

A população inicial é gerada aleatoriamente, garantindo diversidade e evitando que o algoritmo comece preso a soluções locais.

A quantidade de indivíduos é adaptativa: até cinco cidades é usada a combinação total (fatorial), e acima disso o tamanho da população cresce em 50% a cada nova cidade adicionada.



Avaliação das Soluções

A qualidade de cada rota é medida pela soma total das distâncias percorridas entre as cidades. O cálculo considera não apenas os deslocamentos intermediários, mas também o retorno ao ponto inicial, garantindo um percurso fechado.

Essa distância total é usada como critério de avaliação quanto menor a soma, melhor a rota. Com base nesse valor, o sistema define o “fitness” de cada indivíduo e o utiliza nos processos de seleção e cruzamento.



Processo Evolutivo

A cada ciclo, ou geração, o algoritmo realiza uma série de operações inspiradas na evolução natural. Primeiro, os melhores indivíduos são selecionados para reprodução. Em seguida, pares de rotas são combinados para gerar novos indivíduos, e pequenas mutações são aplicadas para introduzir diversidade.

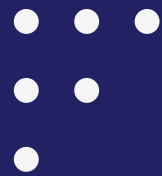
Esse processo se repete até que o algoritmo atinja o limite de estagnação quando a melhor rota deixa de melhorar por várias gerações consecutivas. Dessa forma, o programa garante que o resultado final é obtido por mérito evolutivo, não por simples sorte aleatória.



Seleção e Elitismo e Cruzamento

A seleção dos indivíduos foi implementada pelo método de torneio, no qual três rotas são escolhidas aleatoriamente e a melhor delas é selecionada como “pai”. Além disso, foi aplicado o elitismo, garantindo que o melhor indivíduo de cada geração seja preservado para a próxima.

Para o cruzamento, foi adotado o método Order Crossover (OX), que preserva a ordem relativa das cidades e evita repetições. O processo consiste em escolher dois pontos de corte dentro da rota de um dos pais, copiar o trecho entre eles e preencher o restante com as cidades do outro pai, na ordem em que aparecem.

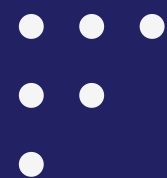


Mutação

A mutação foi tratada de maneira simples e eficaz, utilizando o método swap ou troca de posições.

Durante essa etapa, duas cidades aleatórias trocam de lugar na rota, alterando parcialmente o caminho sem comprometer a validade do indivíduo.

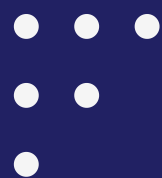
A taxa de mutação é configurável e, por padrão, definida em 5%.



Substituição e Sobrevivência

Após o cruzamento e a mutação, é formada uma nova população. O indivíduo mais apto da geração anterior é mantido (elitismo), enquanto o restante é substituído pelos novos filhos.

Esse mecanismo assegura que as melhores soluções sejam sempre preservadas e que o processo evolutivo siga em direção à rota mais curta possível.



Conclusões

O problema do caixeiro viajante foi tratado com uma abordagem evolutiva capaz de encontrar soluções de boa qualidade sem a necessidade de explorar todas as combinações possíveis.

A modelagem implementada mostra que, mesmo em problemas complexos, é possível alcançar equilíbrio entre desempenho, clareza e precisão.